Special Ship Automation Systems HW:
System Modeling Using Experimental Measurement Data

DAGKLIS SPYRIDON

Supervisor: George Papalambrou

School of Naval Architecture & Marine Engineering
Laboratory of Marine Engineering

-- May 2026

Agenda

1. Introduction & The HIPPO-2 Experimental Facility
2. Data Preprocessing & Statistical Filtering
3. Part 1: Baseline Neural Networks (Constrained Architecture)
4. Part 2: Feature Engineering & Deep Learning Optimization
5. Comparative Analysis (Physical Units Evaluation)
6. Conclusions & Engineering Implications

Time aprox 30 min

W&B

GitHub

LinkedIn

TQRCG

TQRCG

TQRCG

Introduction : The HIPPO-2 Facility & Objectives

Experimental Setup: Hybrid diesel-electric propulsion system (261 kW ICE, 90 kW EM) on a common shaft.

The Engineering Challenge: Real-time measurement of emissions and specific performance metrics is sensor-intrusive, expensive, and prone to failure.

Project Objectives:
Virtual Sensors: Software-based estimation of unmeasured physical parameters.

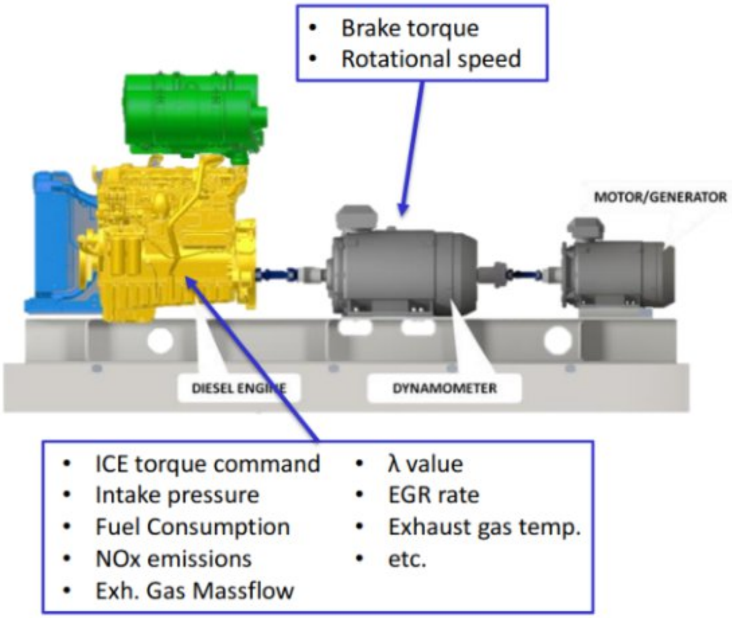
Digital Twin: A robust computational replica for simulation, predictive maintenance, and fault diagnosis.

Methodology: Artificial Neural Networks (ANNs).

Introduction : Prediction Targets & Input Features

Input Features (Predictors):

Operational: Brake Torque (Nm), Rotational Speed (RPM)
Control: ICE Torque Command (%), EGR Rate (%)
Thermodynamic: Exhaust Gas Temp (°C), Exhaust Gas Mass Flow (kg/s)



Part 1: Baseline Neural Network (Shallow)

Prediction Targets: NOx Emissions, Fuel Consumption, Intake Pressure, **λ** value
Architecture: Strictly 1 Hidden Layer | Max 20 Neurons
Focus: Evaluating basic mapping capabilities in shallow network

Part 2: Advanced Neural Network (Deep) - Feature Engineering

Prediction Targets: NOx, Fuel Consumption, Intake Pressure, **λ** value
Architecture: 1-2 Hidden Layers | Up to 100 Neurons per layer
Focus: Addressing complex non-linearities via Feature Engineering and Deep Learning.

Data Preprocessing & Statistical Filtering

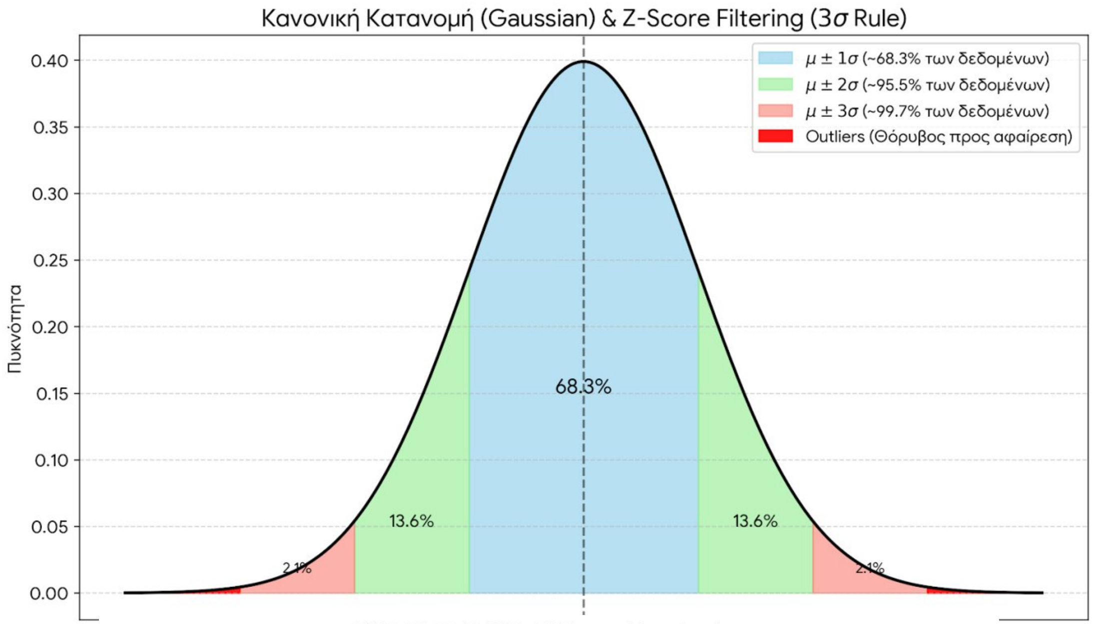
Initial Dataset Overview:
451,228 raw experimental samples.

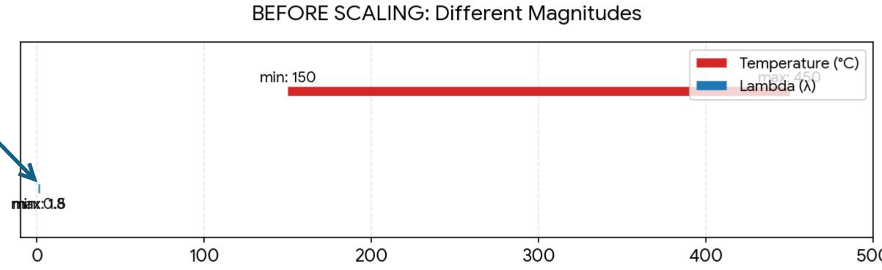
Step 1: Physical Constraint Filtering
Ensuring thermodynamic and chemical validity.
Exclusion of **Nan** values
Exclusion of invalid (**negative**) readings:
Nox ≥0,Fuel ≥ 0, Intake Pressure > 0.

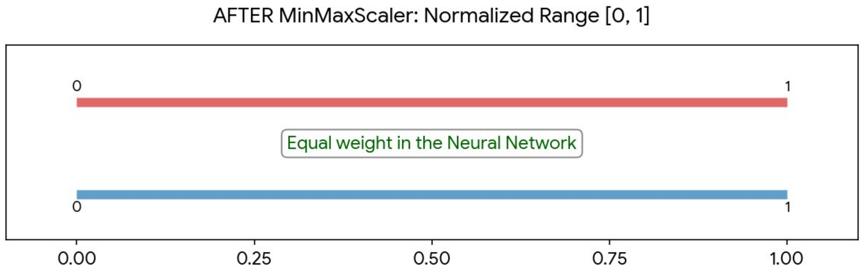
Step 2: Statistical Outlier Removal (Z-Score)
Implementation of the **σ** rule to mitigate sensor noise.
Selection Criteria: **$\mu-3\sigma \leq X \leq \mu+3\sigma$** **μ**: mean **σ**: deviation

Step 3: Data Transformation (MinMaxScaler)
Feature scaling to the [0, 1] range for numerical stability.

Final High-Fidelity Dataset:
428,491 clean samples.
Rejection Rate: 5,4% (common)







Part A: Feature Analysis — Correlation Heatmap & NOx Thermodynamics

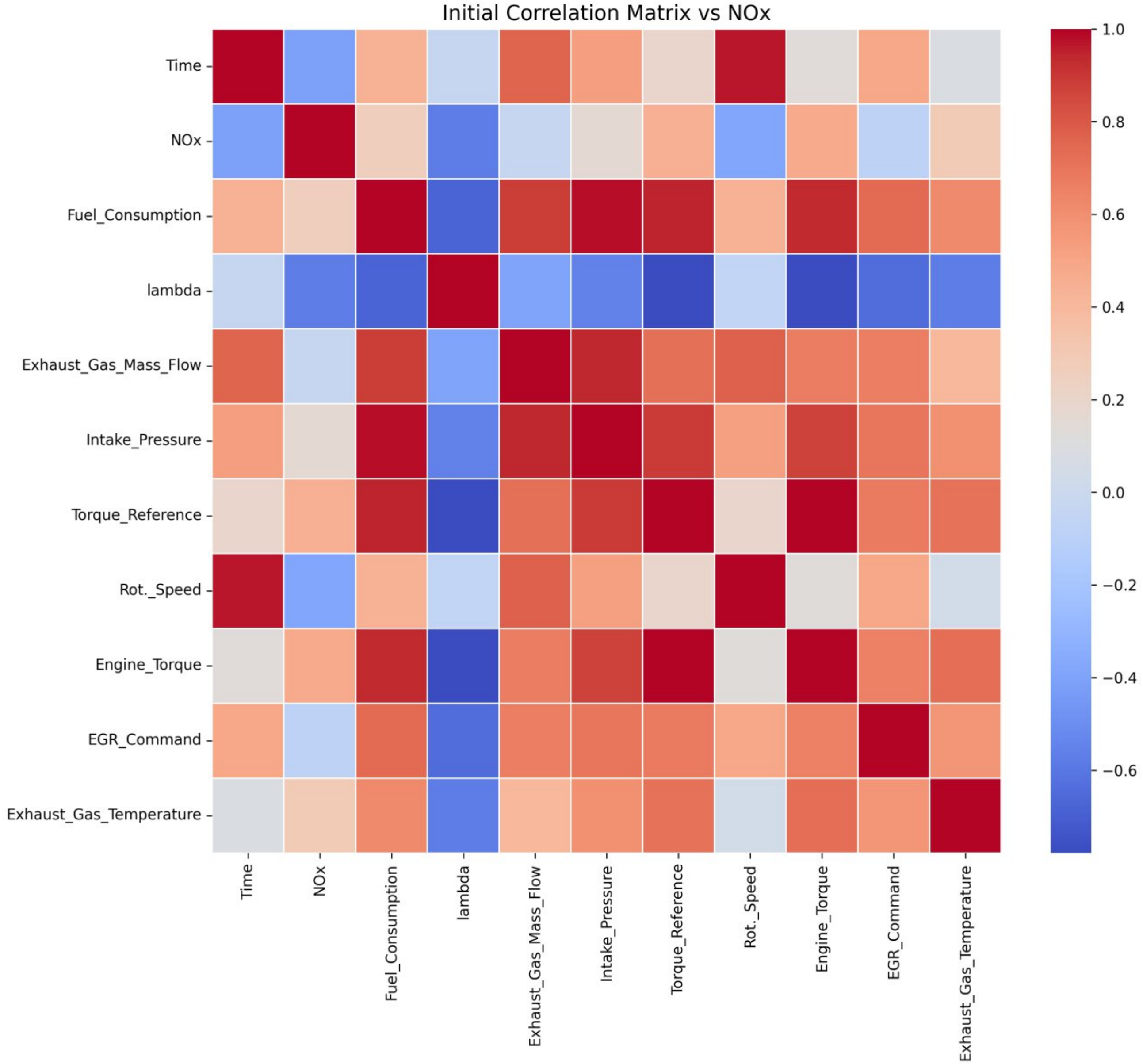
Pearson Correlation Matrix: Mapping linear relationships across the HIPPO-2 dataset.

Strong Predictors: High correlation between *Torque Reference* and *Fuel Consumption* (ρ > 0.9).

Inverse Relationships: *EGR Command* shows a strong negative correlation with *NOx emissions*, aligning with expected engine thermodynamics.

Multicollinearity Check: Ensured selected features provide distinct information to the neural network.

Data and features received in CSV FORMAT.
1. Thermal NOx (Zeldovich): Strong positive correlation with Exhaust Temp — high temperatures trigger exponential NOx formation.
2. EGR Suppression: Strong negative correlation — EGR lowering peak cylinder temps.
3. Non-linear challenge: NOx peaks at slightly lean mixtures — motivates deep learning over shallow approach.



Part A: Exploratory Data Analysis (how data is acquired)

Engine Operating Map: Visualization of experimental data points.

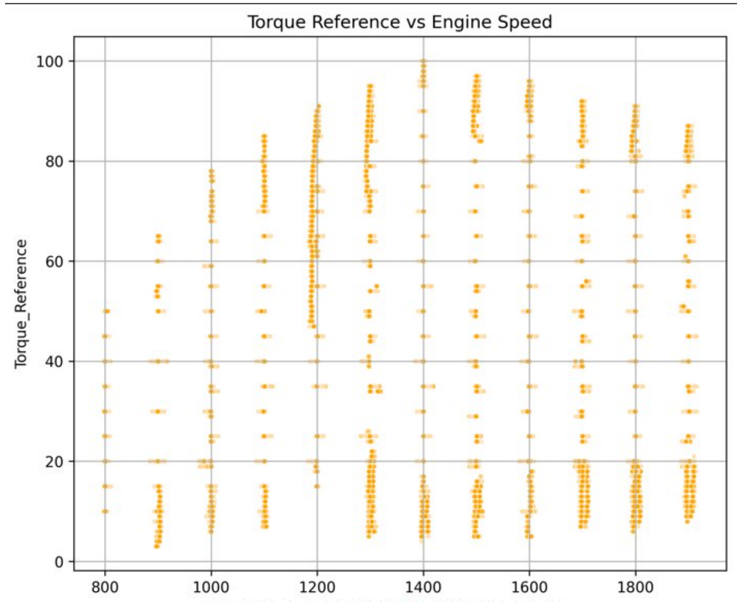
Upper Plot: Torque Reference (%)
Control Signal (Setpoint).Scale: 0 to 100% of maximum engine load.

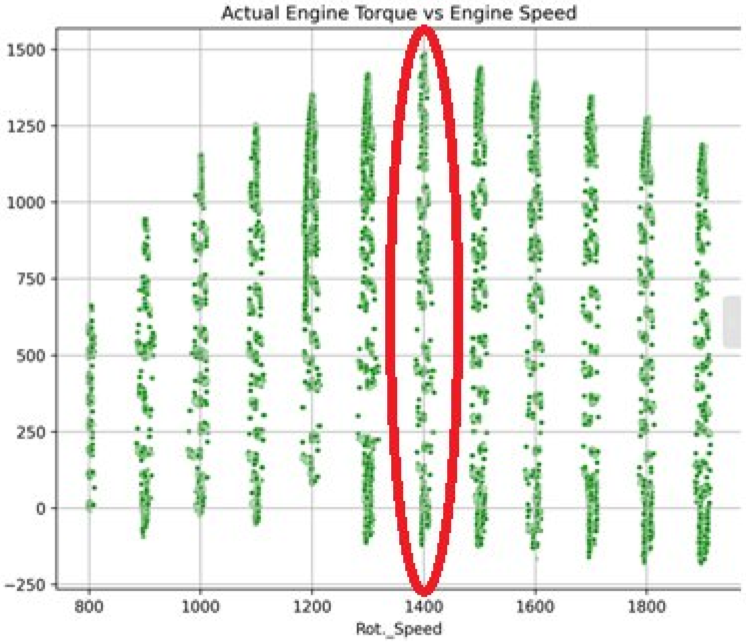
Lower Plot: Actual Engine Torque (Nm)
Physical Response @ shaft.
Range: -250 to 1500 Nm, including transient and **braking** negative phases.

Data Generation Process (Vertical Columns):

Constant Speed Steps: The experiment was conducted at discrete RPM intervals (e.g., every 100 RPM).
Steady-State Mapping: At each fixed speed step, the engine load was varied across its range, creating the vertical distribution of data points.
Result: A structured grid that facilitates the **"Leave-One-RPM-Out"** validation strategy. Use 1400rpm for test train on other rpm sets.

Observation: Data follows a grid pattern, indicating steady-state testing at discrete RPM intervals (800-1900 rpm with step 100rpm).





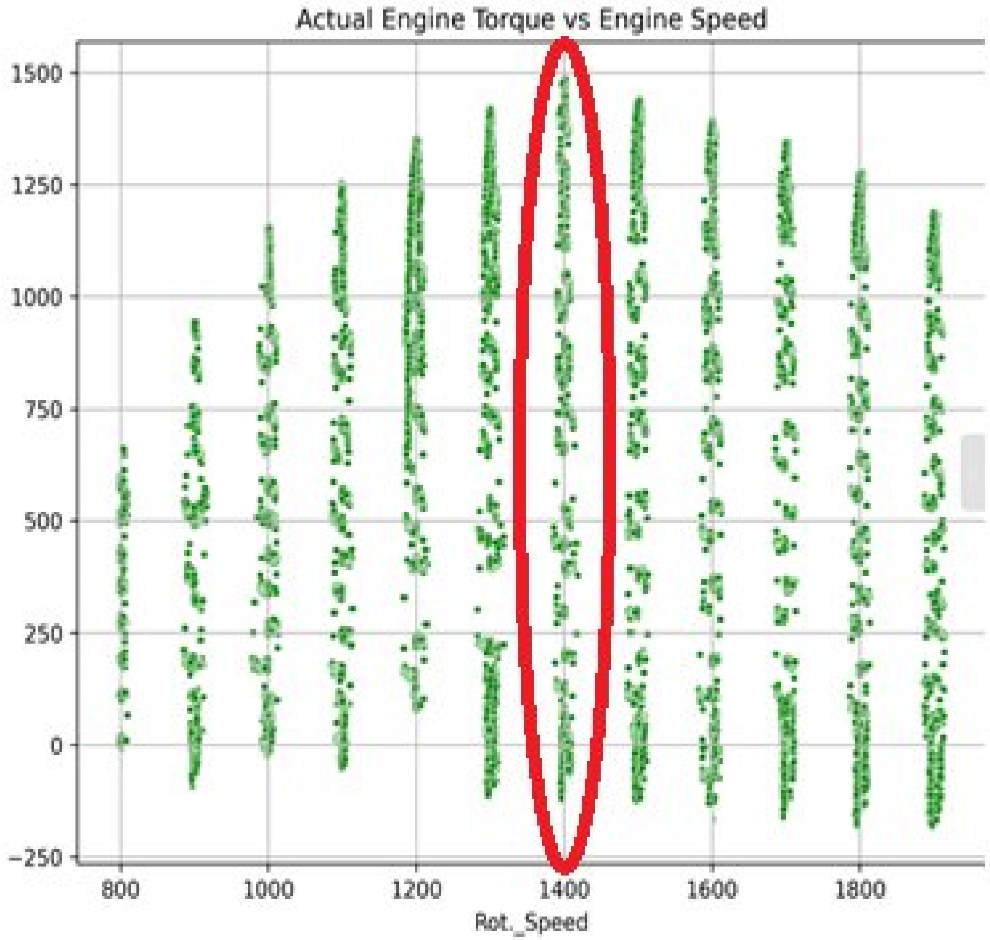
Part A: Data Splitting & Integrity Strategy

The Standard Pitfall: Randomly splitting time-series or steady-state engine data (train_test_split) leads to severe **Data Leakage**. The model memorizes highly correlated adjacent data points.

Our Engineering Approach:
We implemented a "Leave-One-Operating-Condition-Out" strategy.

Execution:
All measurements corresponding exactly to **1400 RPM** were completely isolated and reserved exclusively for the **Test Set**.
The remaining RPM ranges (Train/Val sets) were then randomized to ensure training stability.

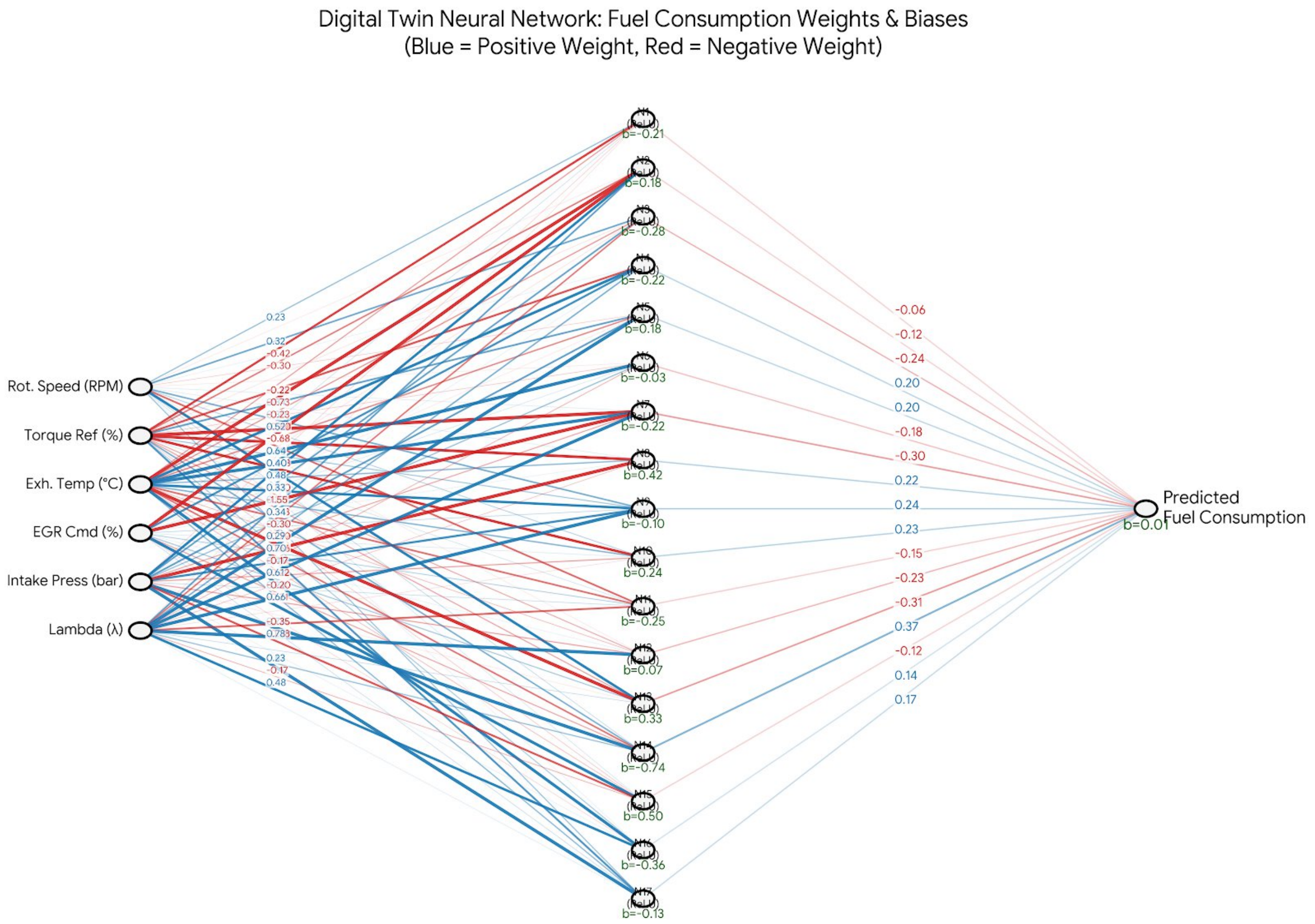
Objective: Force the Neural Network to interpolate and predict behavior at an unobserved engine speed.
Integrity Measures: Time column dropped (physics mapping, not temporal). Scaler fit only on training data — test set has zero prior knowledge. DataLoader shuffled to break sequential bias.

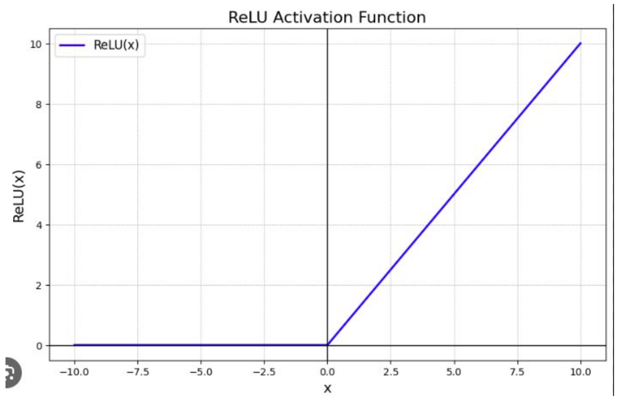


Part A Architecture – The Baseline Neural Network

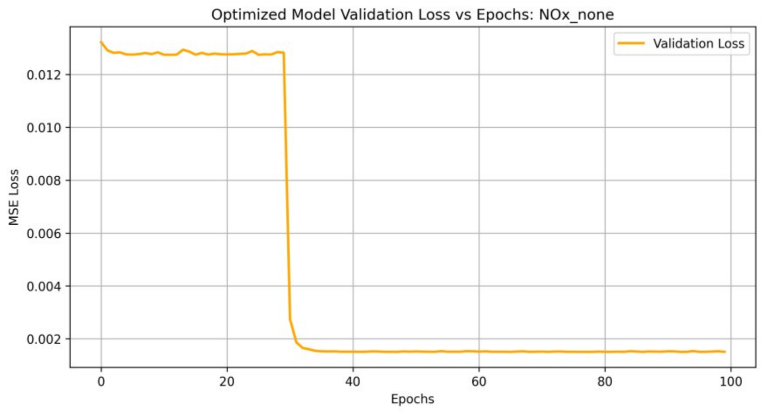
Network Constraints (Task 4):
Hidden Layers: 1.
Neurons: Maximum 20.
Hyperparameter Optimization Optuna for automated search.
Neurons (1-20) and L. Rate from 10^-4 to 10^-6

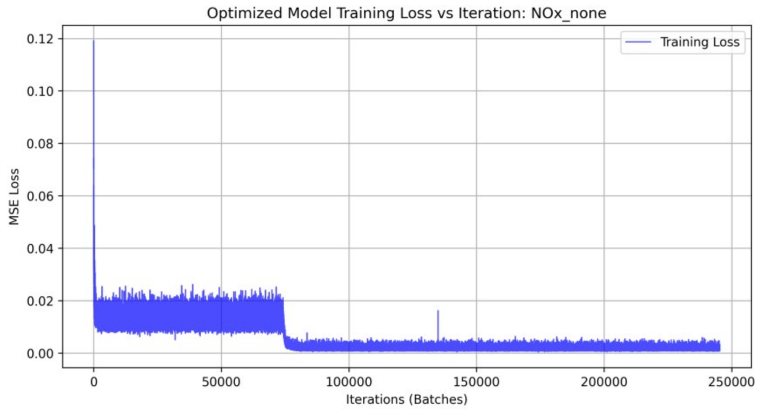
Optimizer: Adam Optimizer.
Activation Function: ReLU(Rectified Linear Unit).
Training Duration: 100 Epochs with MSE loss.
Target Parameters:
NOx and **Fuel Consumption**





Part A: The Challenge of Modeling Nox shallow network

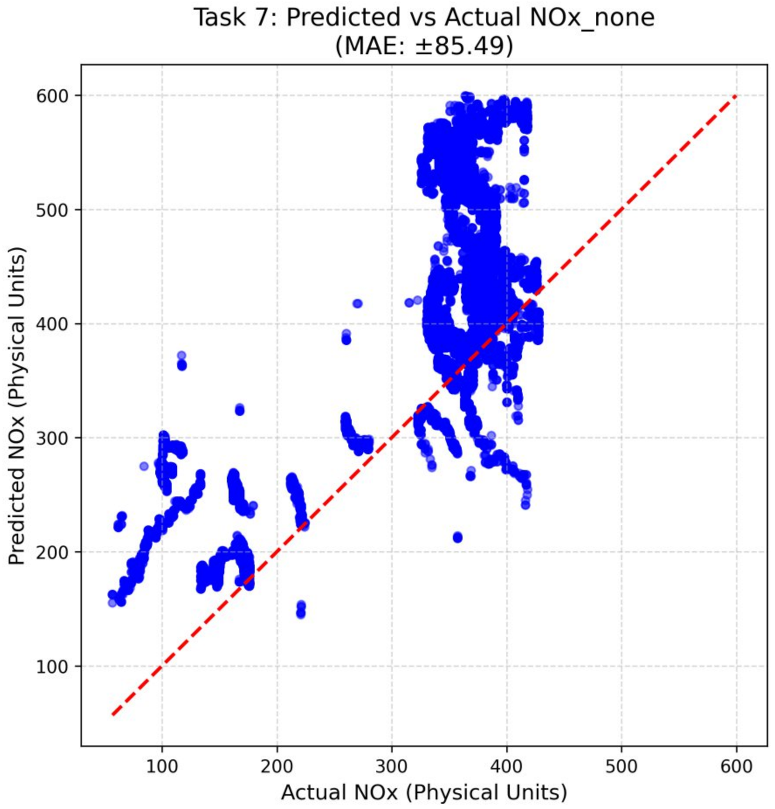




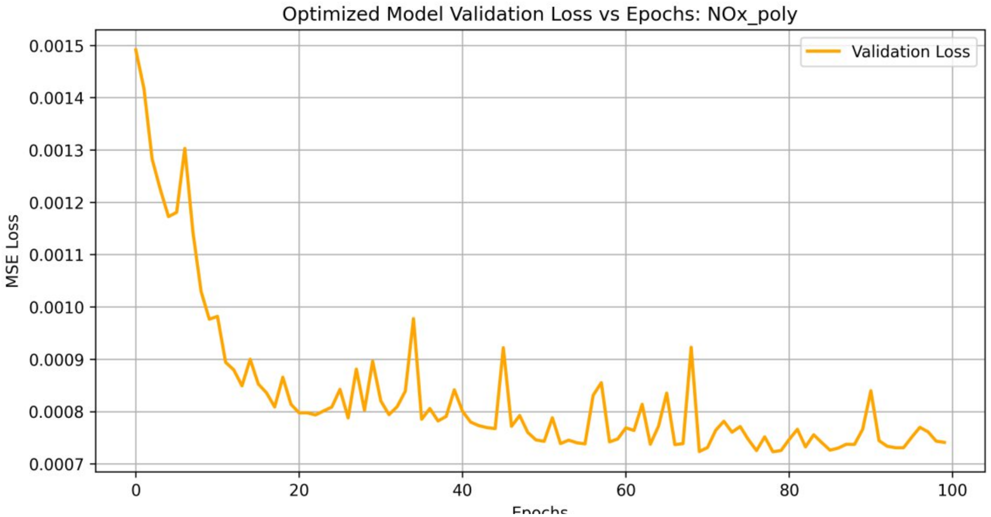
Training Dynamics
Smooth convergence achieved (MSE stabilizes around 5400).
Indicates successful optimization via Optuna, avoiding numerical instability.
Prediction Performance
$R^2 = 0.5898$
MAE = ± 84.49**ppm**.(Mean Absolute Error)
Significant dispersion in the "Predicted vs Actual" scatter plot.

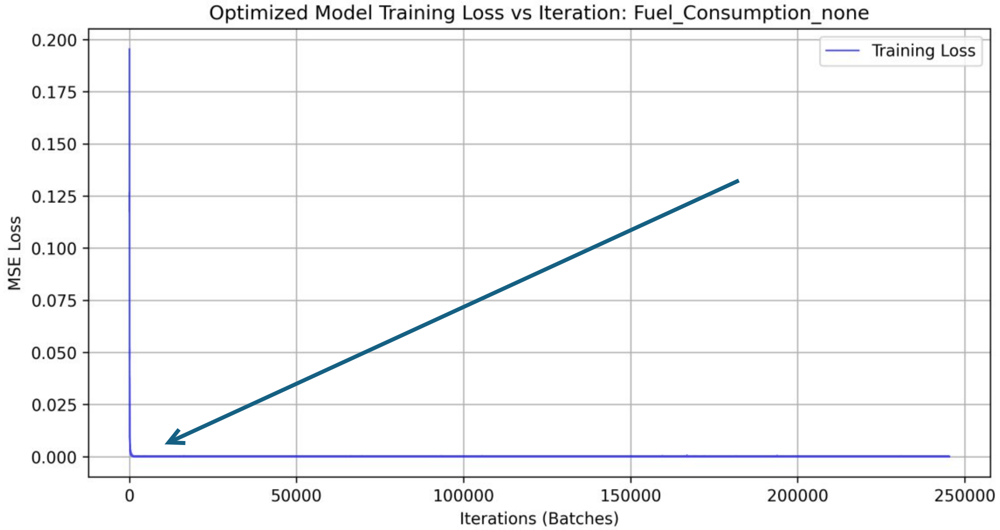
Engineering Insight: The baseline model exhibits **underfitting**. It lacks the representational capacity to capture the highly non-linear, exponential thermal dynamics of NOx formation (Zeldovich mechanism).

ADAM and OPTUNA worked well



Part A: Modeling Fuel Consumption

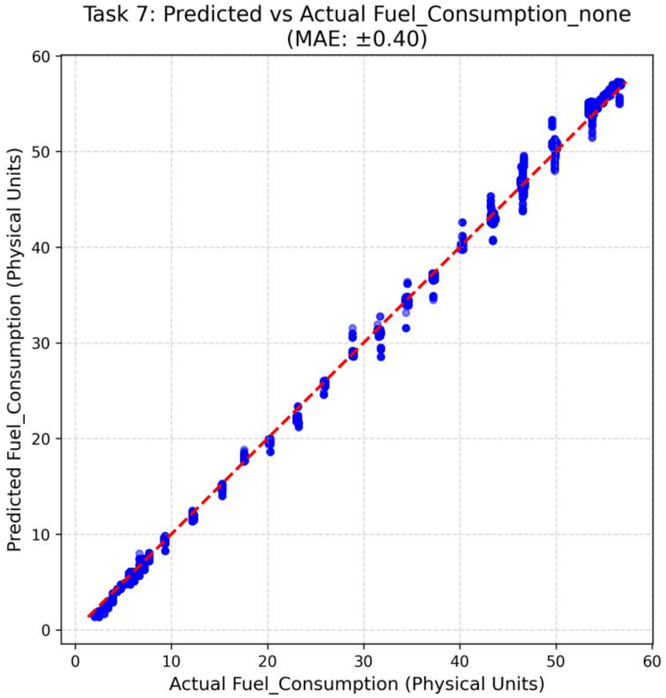




Training Dynamics (Left Plots):
Rapid, smooth convergence to near-zero(MSE).
No signs of overfitting; training and validation curves align perfectly.

Prediction Performance (Right Plot):
$R^2 = 0.9926$ | MAE = ± 0.4 kg/h.
Near-perfect alignment along the y=x ideal diagonal.

Engineering Insight: The baseline shallow network (20 neurons) is highly capable of modeling the linear relationship between engine load/torque and fuel mass flow.



Part B Strategy for Model Optimization

The Challenge: Overcoming the "Underfitting" of the baseline NOx model R^2 approx 0.59).

Strategy 1: Advanced Feature Engineering

Method: Symbolic Regression (Genetic Programming via gplearn).
Goal: Automatically discover non-linear transformations and interactions between physical parameters (e.g., Temperature, λ, EGR).

Strategy 2: Enhanced Model Capacity

Depth: Transitioning from 1 to 2 hidden layers.
Neurons: Increasing capacity up to 100 neurons as permitted by the new constraints.

Strategy 3: Hyperparameter Tuning

Using **Optuna** to find the global optimum for Learning Rate, Batch Size, and Layer sizes.

Symbolic Regression via gplearn: Genetic programming evolves mathematical expressions to discover hidden non-linear relationships between raw sensor data (e.g., polynomial interactions, parameter ratios). The NN receives pre-computed “thermodynamic hints” rather than raw scalars.

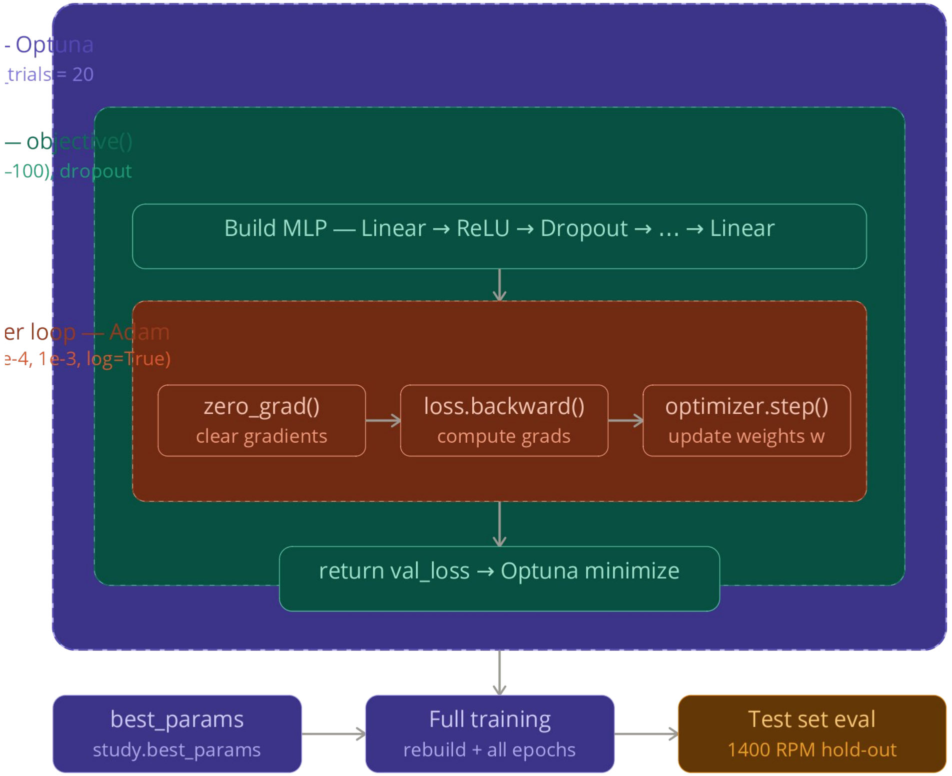
Three feature modes tested per target: none (raw), poly (polynomial interactions), gplearn (symbolic regression) — winner selected by test MAE.

Part B: Feature Selection via Mutual Information

Why Mutual Information (MI)? Pearson correlation only measures linear relationships. MI captures any statistical dependency — including the complex non-linear interactions in gplearn-generated features that Pearson would miss.
Method: mutual_info_regression(X_train, y_train) scores each feature by Information Entropy. Top 10 features selected per target, keeping only the most informative predictors for the final model.
Dynamic Architecture Assembly: PyTorch nn.Sequential is built dynamically from Optuna's best hyperparameters — number of layers, neurons per layer, and dropout rates are all configured automatically from study.best_params.
Outcome: MI ensures the final feature set contains maximum predictive information. Combined with Optuna's architecture search, this pipeline is fully automated — no manual feature engineering guesswork.
Literature: Optuna [Akiba et al., 2019] — WandB [Biewald, 2020]

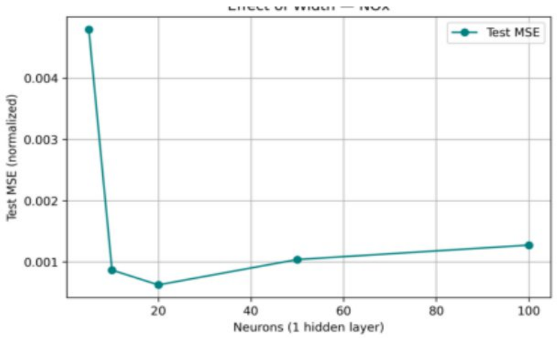
Two-Level Optimization: Optuna Searches, Adam Learns

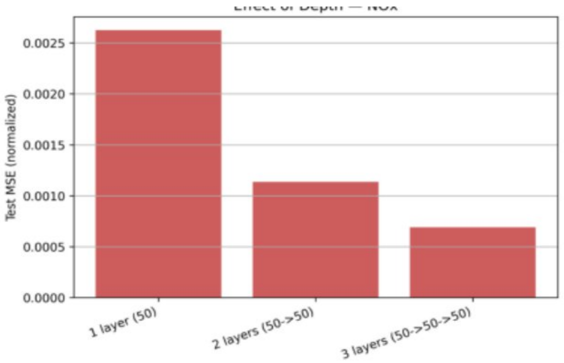
OUTER LOOP — Optuna (Hyperparameters)
study.optimize(objective, n_trials=20) — 20 trials, each calling objective():
Optuna proposes: learning rate (log-uniform 1e-4 to 1e-3), #layers (1–2), #neurons (10–100), dropout.
Build MLP with those hyperparameters (Linear → ReLU → Dropout → …).
Train briefly with Adam → return val_loss to Optuna.
Bayesian TPE uses past trial results to focus on promising regions — smarter than grid/random search.
INNER LOOP — Adam (Weights)
Per batch: zero_grad() → loss.backward() → optimizer.step(). Adam updates weights with momentum + per-parameter adaptive step size. Uses the learning rate chosen by Optuna.
The link: optim.Adam(model.parameters(), lr=trial.suggest_float('lr', 1e-4, 1e-3, log=True))
Final step: best = study.best_params → rebuild + fully train the winner → evaluate on held-out 1400 RPM test set.
Adam asks "how do I change the weights?"; Optuna asks "which lr/architecture do I give Adam?" — Optuna chooses, Adam applies.



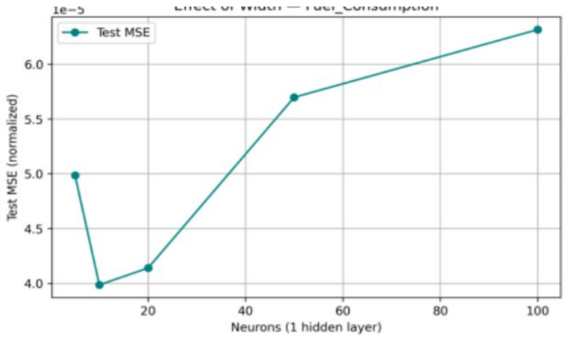
Architecture Ablation — Each Change vs. Test Loss

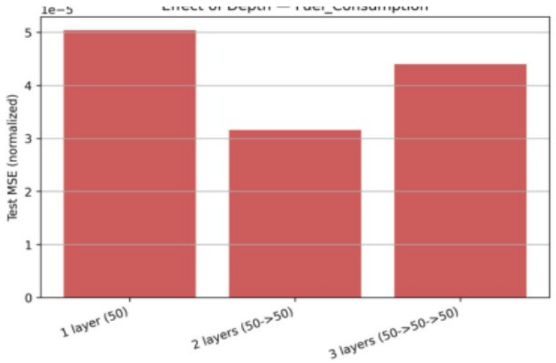
NOx Ablation (none mode, normalized test MSE)
Width study (1 layer, varying neurons):
5 neurons: 0.00130
10 neurons: 0.00065
20 neurons: 0.00074
50 neurons: 0.00056
100 neurons: 0.00052 ← best width
Depth study (50 neurons per layer):
1 layer: 0.00060 ← best depth
2 layers: 0.00070
3 layers: 0.00076 (diminishing returns)
Conclusion: more neurons → lower error; more layers → mild overfitting beyond 1–2.
Optuna found 2-layer (97→65) with raw features as global winner.





Feature Mode Summary (test MAE, best architecture per mode)
NOx Emissions:
none (raw features, 2L 97→65):   13.56 ppm ✓ BEST
poly (2L 94→96):   17.91 ppm
gplearn (2L 79→100):   18.38 ppm
Part A baseline (1L ≤20 neurons): 84.49 ppm
Network depth fixed NOx underfitting; engineered features did NOT help.
Fuel Consumption:
poly (1L 40 neurons):   0.285 kg/h ✓ BEST
gplearn (1L 100 neurons):   0.292 kg/h
none (raw features, 1L 89):   0.345 kg/h
Part A baseline (1L ≤20 neurons): ~0.40 kg/h
Feature engineering DID help Fuel — polynomial interactions capture load–temperature coupling.
Unified message: Capacity solved the non-linear target (NOx); feature engineering helped the near-linear one (Fuel). We followed the data, not an assumption.





*Ablation =Froze everything except one variable at a time to isolate the effect of each design choice."

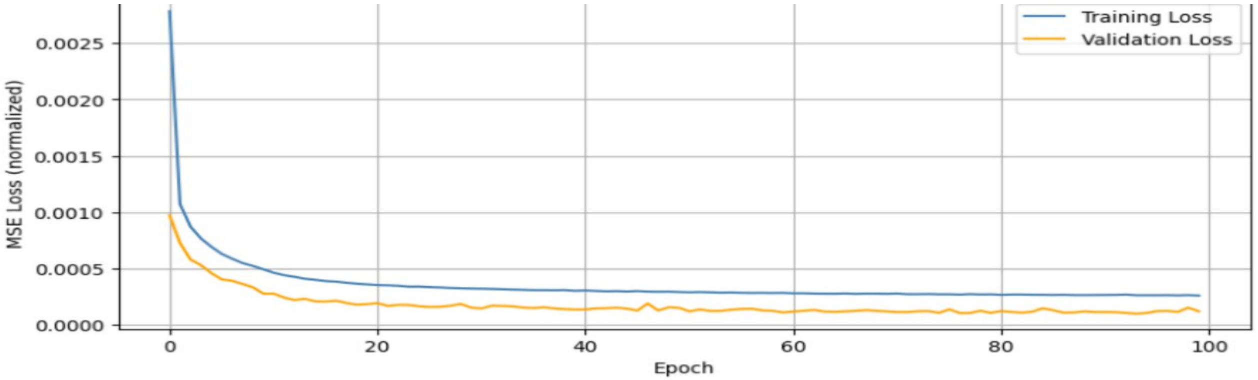
Part B Results — NOx Emissions: Deep Network vs. Baseline

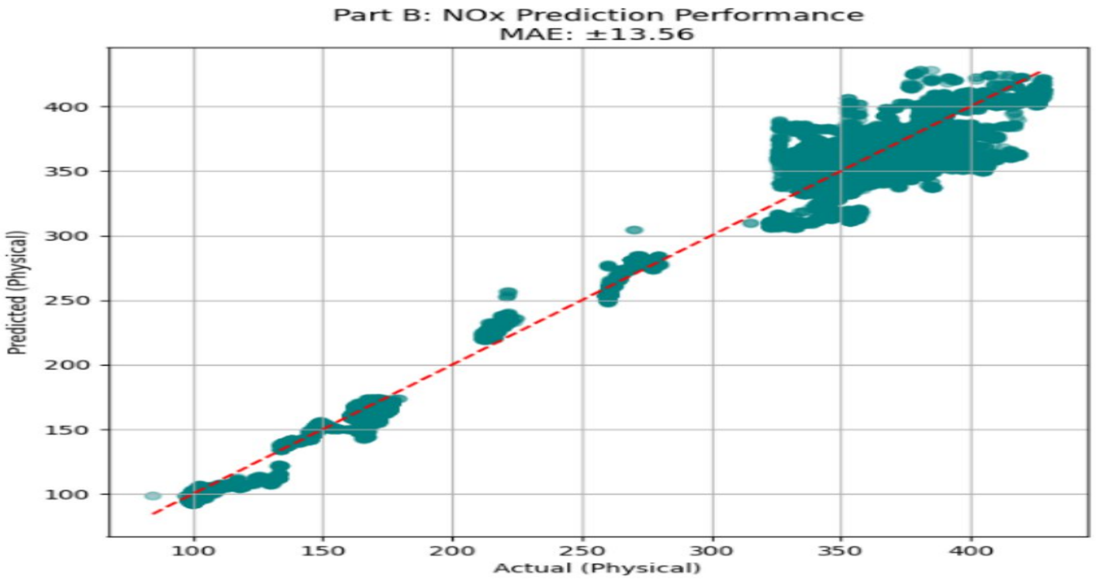
Target: NOx Emissions (ppm) – The Non-Linear Challenge.
Training Dynamics: Stable convergence despite increased network depth, thanks to precise
Optuna learning rate tuning.

Performance Breakdown:

Part A (Baseline): Massive dispersion, severe underfitting.
-Mean Absolute Error (MAE): ± 84.49 ppm
Part B (Deep + Raw Features (none mode)): Dramatic reduction in variance. Predictions tightly align with the ideal y=x diagonal.
- Mean Absolute Error (MAE) = ±13.56 ppm

Engineering Impact: A 2-layer network with raw features (MAE ±13.56 ppm) solved underfitting. Network depth — not feature engineering — was the key improvement for NOx.



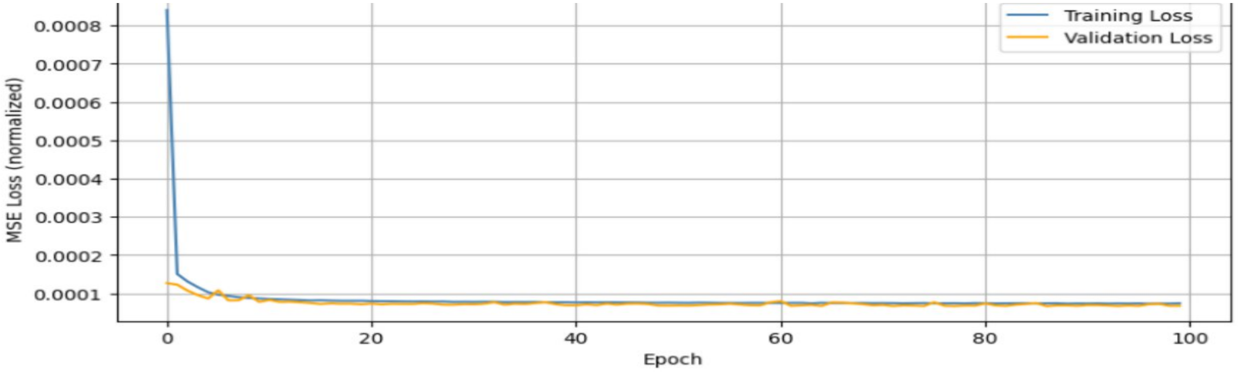


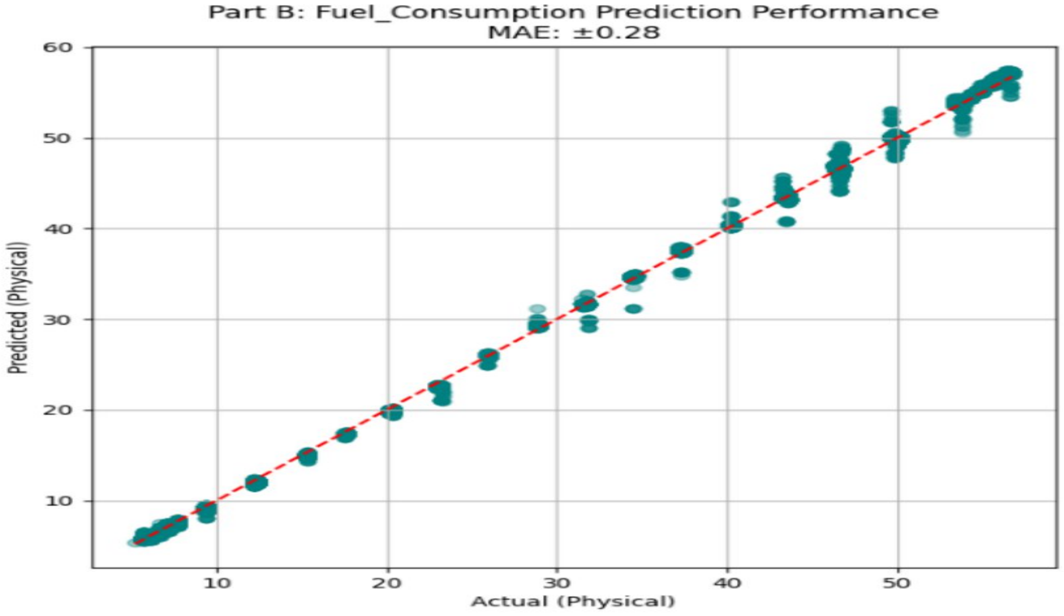
Part B Results Fuel Consumption Feature Engineering vs None

Target: Fuel Consumption (kg/h).
Training Dynamics: Rapid convergence.
 Validation loss tightly tracks Training loss Zero Overfitting.

Performance: Near-perfect predictive capability
(R^2 =0.999594). MAE ± 0.285

Optuna : 1 Layer (40 neurons)
Conclusion: The optimized shallow network (40 neurons) confirms that fuel dynamics scale directly and predictably with mechanical load (Torque/RPM).





Model Interpretability with SHAP — The Model Learned the Physics

What is SHAP? Shapley values (game theory) provide fair, per-feature attribution for each prediction. [Lundberg & Lee, 2017]

NOx best model: none (raw features, 2L 97→65). Mean |SHAP|:

Rot. Speed: 0.166 ← #1
Engine Torque: 0.089
Exh. Gas Temp: 0.026
Torque Ref: 0.015
EGR: 0.010

Physics check: Speed/load drive Zeldovich thermal NOx ✔

NOx gplearn SHAP: raw features rank ABOVE engineered ones (Engine_Torque 0.229 > EGR 0.183 > Exh_Temp 0.088 > GP_3 0.078, GP_4 0.070, GP_1 0.056). Engineered features did NOT contribute meaningfully — consistent with gplearn (18.38 ppm) losing to none (13.56 ppm).

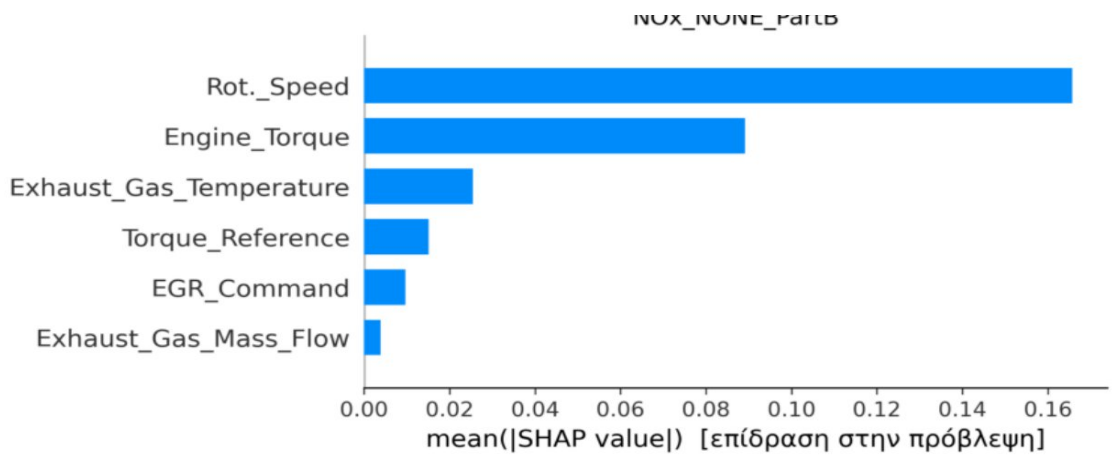
Fuel best model: poly (1L 40 neurons). Mean |SHAP| top features:

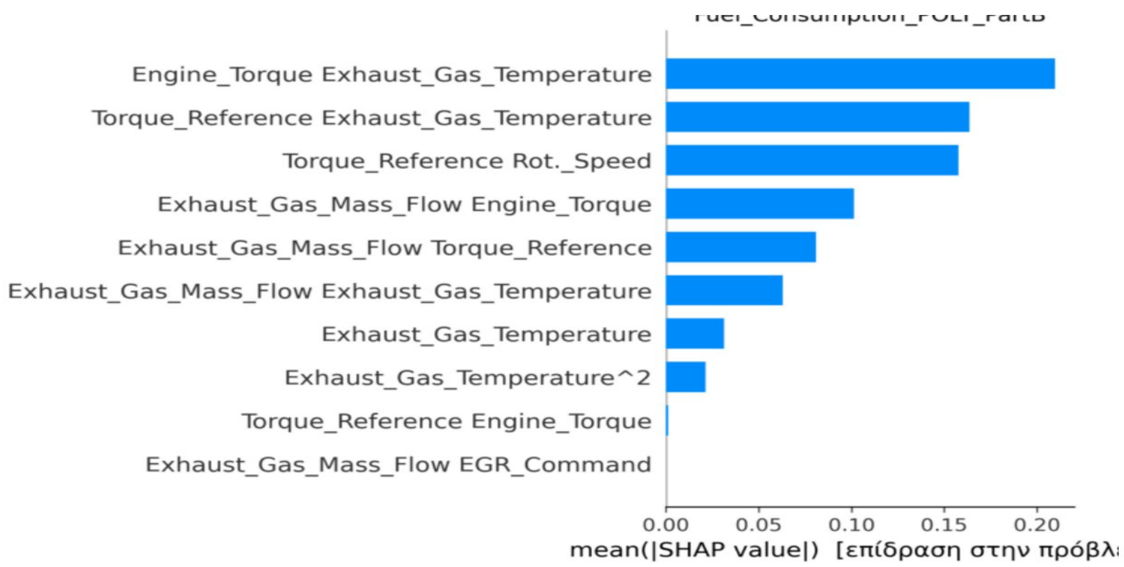
Torque×Exh_Temp (poly): 0.210 ← #1
TorqueRef×Exh_Temp (poly): 0.164
TorqueRef×Rot_Speed (poly):0.158
Engine Torque (raw): 0.239

Engineered polynomial interactions ARE the top predictors for Fuel ✔ Explains why poly beats none (0.285 vs 0.345 kg/h).

Key finding: Feature engineering helps when the physical relationship has exploitable interaction structure (Fuel: load–temperature coupling). It does NOT help when the problem is network capacity (NOx: non-linear dynamics needing depth, not new features).

Improvement summary: NOx: 84.49 → 13.56 ppm (~6× better). Fuel: ~0.40 → 0.285 kg/h. SHAP confirms physically meaningful feature use throughout.





***SHAP (**Shapley Additive exPlanations**)** witch feature contribues most

Part A vs Part B — Results Comparison

Target Parameter	Part 1 (Shallow + Raw Features)	Part 2 (Deep + best features)	Accuracy Improvement
NOx Emissions	MAE: 84.49 ppm	**MAE: 13.56 ppm**	6× better — depth fixed underfitting, not features
Fuel Cons.	MAE: 0.40 kg/h	MAE: 0.285 kg/h	Poly features helped near-linear target
Intake Pressure	(Not modeled)	**MAE: 1.21 mbar**	Lab-Grade Precision
Lambda	(Not modeled)	**MAE: 0.039**	Wideband Sensor Eq.

NOx — network depth, not features: A 2-layer network with raw features achieved MAE 13.56 ppm (~6× better than baseline). Poly and gplearn features were worse (17.91, 18.38 ppm). Depth solved underfitting.

Fuel — features helped: Near-linear target achieved $R^2 = 0.9995$ with 1 hidden layer + polynomial interactions (0.285 kg/h). Optuna correctly avoided over-engineering.

Leakage-free by design: Test set = 1400 RPM, completely unseen during training and hyperparameter search — not inferred from R^2 scores.

Engineering Conclusions & Future Integration

Virtual Sensors Validated:
The trained neural networks successfully mapped the thermodynamic states of the HIPPO-2 facility. Achieved errors (e.g., λ 0.039) fall within the standard tolerance of expensive physical sensors.

Methodological Success:
Feature Engineering: Poly features improved Fuel (MAE 0.285 kg/h). For NOx, a 2-layer network with raw features was sufficient — engineered features were not needed.

Optimization: Bayesian search (Optuna) effectively balanced network depth with generalization (dropout).

Future Application (The Digital Twin):
Models are lightweight and ready for edge-computing deployment.
Can operate in parallel with physical sensors for **real-time fault diagnostics** (Anomaly Detection) and **CII compliance monitoring** on autonomous vessels.

Key Results: NOx: 84.49 → 13.56 ppm (~6× improvement, depth solved underfitting). Fuel: 0.40 → 0.285 kg/h (poly features helped the near-linear target). $R^2 > 0.99$ all targets. Leakage-free validation (1400 RPM held out).
Future Work: (1) Edge deployment via ONNX for shipboard IoT. (2) LSTM networks for transient (time-dependent) engine behavior. (3) Continuous online retraining as engine ages. (4) Anomaly detection and CII compliance monitoring integration.